

Reanimating the First AI: The Logic Theorist Runs Again in IPL-V

Jeff Shrager, Ph.D.
Bennu Climate, Inc.
jeff@shrager.org

Abstract—The Logic Theorist (LT), created by Allen Newell, J.C. (Cliff) Shaw, and Herbert Simon at the RAND Corporation and Carnegie Tech in 1955–1956, is widely regarded as the first artificial intelligence program. LT was implemented in the Information Processing Language (IPL), a list-processing language that directly foreshadowed Lisp. Unlike Lisp, however, IPL has been essentially extinct for over half a century; no implementations are known to have survived, and no community of practitioners remains. Here I describe the construction of a new IPL-V interpreter, written in Common Lisp, and the faithful reanimation of the original Logic Theorist from code transcribed directly from Stefferud’s 1963 RAND technical report. The reanimated LT successfully proves 16 of 23 theorems from Chapter 2 of Whitehead and Russell’s *Principia Mathematica*, results that are historically consistent with the original. This effort continues a program of reanimating foundational AI systems on their original platforms, following the recent restoration of Weizenbaum’s ELIZA on a reconstructed CTSS. A distinctive aspect of this project was the essential role played by large language models (LLMs) as collaborative debugging partners—learning an essentially dead programming language alongside the author, and ultimately proving indispensable in navigating thousands of lines of execution traces from a heuristic program whose correctness could not easily be verified.

Index Terms—Logic Theorist, IPL-V, artificial intelligence history, software archaeology, Newell, Simon, Shaw, list processing, digital preservation

I. INTRODUCTION

In the summer of 1956, a small group of researchers gathered at Dartmouth College for a workshop on what John McCarthy had dubbed “artificial intelligence.” Among them were Allen Newell and Herbert Simon, who arrived with something none of the other attendees had: a working program that could discover proofs of mathematical theorems. The Logic Theorist (LT) had already proved 38 of the first 52 theorems in Chapter 2 of Whitehead and Russell’s *Principia Mathematica* [1], in some cases finding proofs more elegant than those produced by the human authors. It was, arguably, the first artificial intelligence.

LT was implemented in the Information Processing Language (IPL), a language created by the same team—Newell, Shaw, and Simon—specifically for building cognitive models [2]. IPL introduced list manipulation, dynamic memory allocation, recursion, and higher-order functions. In a meaningful sense, it was the direct ancestor of Lisp [3]. Yet while Lisp has thrived for nearly seven decades, IPL has been extinct since the mid-1960s. No implementations are known to survive, and to the author’s knowledge, no one has previously

attempted to build an IPL-V interpreter and run the original AI programs.

This paper describes the construction of a complete IPL-V interpreter in Common Lisp, and the faithful reanimation of the Logic Theorist from code transcribed directly from Stefferud’s 1963 RAND technical report [4]. More than 99% of the LT code running in this reconstruction is the original code from that report. The reanimated LT proves 16 of 23 attempted theorems from *Principia Mathematica*, with results that are historically consistent with the original system’s known behavior.

This work continues a program of reanimating foundational AI and cognitive science programs on their original platforms. In prior work, a team including the present author restored Weizenbaum’s original ELIZA to operation on a reconstructed CTSS running on an emulated IBM 7094 [6]. The present project differs in both method and character: where the ELIZA reanimation reconstructed an entire hardware and operating system stack, the LT reanimation emulates the abstract IPL-V machine in its direct descendant language, Common Lisp. And where ELIZA was a team effort, the LT project was largely a solo endeavor—albeit one in which large language models played an unexpectedly essential role as collaborative debugging partners.

II. HISTORICAL BACKGROUND

A. The Origins of the Logic Theorist

The story of the Logic Theorist begins in the early 1950s at the RAND Corporation in Santa Monica, California. Herbert Simon, a political scientist consulting at RAND, saw a printer rendering a map using ordinary characters and realized that a machine capable of manipulating symbols could simulate decision-making—and perhaps thought itself. Allen Newell, a RAND scientist studying logistics and organization theory, had a complementary epiphany watching Oliver Selfridge present his work on pattern recognition in 1954: he saw how the interaction of simple programmable units could produce complex, intelligent behavior.

Newell and Simon began collaborating on a program that could prove theorems in propositional logic, enlisting RAND programmer J.C. (Cliff) Shaw to handle the implementation. The first version was famously hand-simulated: Simon assembled his wife and children, gave each a card representing a component of the program, and had them execute the algorithm by passing cards back and forth.

By 1956, the Logic Theorist was running on the JOHNNIAC [7], one of the first Turing–Von Neumann machines, built at RAND and named after Von Neumann himself. The JOHNNIAC had just 4,096 40-bit words of memory. Within these severe constraints, LT proved 38 of the first 52 theorems in Chapter 2 of *Principia Mathematica*, discovering proofs by heuristic search through a space of possible proof steps. For one theorem, LT found a proof more elegant than the one published by Whitehead and Russell.

It is worth noting that Art Samuel’s checkers-playing program existed a couple of years before LT, and it too was heuristic. But Samuel’s goal was to build a checkers player, whereas Newell, Simon, and Shaw’s explicit goal was to model human cognition. They held specific hypotheses about how humans reason and about the “physical symbol system” that, by their hypothesis, underlies intelligence. Their research program spanned many domains over the following decades. This is not to diminish Samuel’s brilliant effort—he also built arguably the first machine learning program, and the first system to learn from self-play.

B. IPL: The First List-Processing Language

To implement LT, Newell, Shaw, and Simon needed a programming language suited to symbolic reasoning. No such language existed, so they created one: the Information Processing Language [2]. IPL went through several versions; IPL-V, released in 1964, was the only version made publicly available [5].

IPL-V runs on an abstract register machine with a push-down stack, working registers (W0–W9), communication registers (H0–H5), and a symbol table of cells. Each cell has three fields: a symbol (SYMB), a link (LINK), and a set of process qualifiers (PQ). Everything in IPL-V—data, routines, even the program itself—is a list of symbols. Routines are invoked by executing their name as an instruction; the machine traverses the list associated with that name, interpreting each cell as an instruction.

The language’s built-in operations are called J-functions (because their names begin with “J”: J0, J1, J2, and so on). There are approximately 150 of these, implementing operations from basic list manipulation to generator protocols (a form of lazy iteration that anticipates modern iterators). In the original IPL-V, many J-functions were themselves written in IPL, although some required machine-level primitives.

IPL is, in a meaningful sense, the direct ancestor of Lisp [3]. It introduced list and symbol manipulation, the idea that programs are themselves lists, dynamic memory allocation, recursion, and higher-order functions. Having now spent months implementing an IPL-V interpreter, I can report that IPL had essentially *everything* that Lisp has, with one critical exception: homoiconic syntax. Where Lisp unified code and data in a single, elegant, parenthesized notation, IPL expressed its ideas in an assembly-language style that, while powerful, was far more difficult to read and write. McCarthy’s genius was not in inventing list processing *de novo*, but in finding the syntactic form that made these ideas accessible. Having programmed in Lisp for fifty years and attributed its

brilliance entirely to McCarthy, building this interpreter gave me a deepened appreciation for Newell, Simon, and Shaw’s foundational contributions.

C. The McCorduck Interviews

Among the historical materials that informed this project were transcripts of interviews conducted by Pamela McCorduck for her pioneering history of AI [8]. These interviews, originally obtained for the ELIZA reanimation project [6], include conversations with Newell, Simon, and Shaw themselves. One passage proved particularly resonant: Shaw describes the difficulty of debugging a heuristic program—one that does not crash, does not produce obviously wrong output, but simply generates vast quantities of behavior that might or might not be correct. As described in Section VI, I encountered precisely this problem, sixty years later, with the same program.

III. SOURCE MATERIALS AND TRANSCRIPTION

Unlike the ELIZA reanimation, which began with the dramatic discovery of original source code printouts in Weizenbaum’s archives at MIT [6], the source materials for the Logic Theorist were already publicly available, if not widely known. Two documents were indispensable:

- 1) **Stefferd (1963)**, *The Logic Theory Machine: A Model Heuristic Program*, RAND Corporation memorandum RM-3731 [4]. This report contains a complete listing of the LT program in IPL-V, along with detailed descriptions of its M-routines (the main heuristic procedures).
- 2) **Newell et al. (1964)**, *Information Processing Language-V Manual*, 2nd edition [5]. The definitive reference for IPL-V, including the semantics of all J-functions, cell structure conventions, and the generator protocol.

The IPL-V manual proved to be an exceptionally well-written document. There were, however, occasional ambiguities—places where the specification assumed familiarity with conventions of 1960s computing (such as BCD character encoding) that are no longer common knowledge.

The LT code was transcribed from Stefferd’s report into a Google spreadsheet by the author and Anthony Hay. Subsequently, Rupert Lane identified several transcription errors using his “gridlock” tool [10], software designed to accurately extract tabular code from PDF documents. Despite multiple people putting multiple sets of eyes on the code, a surprising number of small transcription errors persisted through several rounds of review.

The transcribed code was converted into an S-expression format (.lispv files) suitable for loading by the Lisp-based interpreter. Conveniently, Lisp comment characters work in these files, allowing corrections and annotations to be documented inline.

IV. THE IPL-V INTERPRETER

Rather than reconstructing the original hardware environment (as was done for ELIZA on the IBM 7094 and CTSS), I chose to emulate the IPL-V abstract machine in Common

Lisp. This decision was driven by several considerations. First, the JOHNNIAC on which LT originally ran no longer exists and, to my knowledge, no emulator for it is available. Second, IPL-V is defined as an abstract machine specification, not as a program for a specific computer; it was implemented on many different machines through the mid-1960s before being supplanted by Lisp. Third, there is a pleasing circularity in implementing IPL-V—Lisp’s direct ancestor—in Lisp itself.

The interpreter (`iplv.lisp`) implements the complete IPL-V abstract machine as described in the 1964 manual: the cell table with SYMB, LINK, and PQ fields; the communication registers H0–H5 (where H0 serves as the parameter/result register, H1 as the program counter with its associated call stack, and H2 as the free list); the working registers W0–W9; and the push-down stack. Lists are stored as linked chains of cells, exactly as specified.

The core of the interpreter is a function `ipl-eval`, which fetches and executes instructions by traversing the list named in the program counter. `ipl-eval` is re-entrant, supporting the recursive subroutine calls that IPL-V requires. The J-functions are implemented as Lisp functions dispatched by name. I implemented only those J-functions actually used by LT (and a few test programs), amounting to several dozen of the approximately 150 defined in the manual.

A complete run of LT—attempting proofs of 23 theorems—takes approximately 574,000 IPL machine cycles and finishes in a few seconds on modern hardware. On the JOHNNIAC at RAND in 1956, it likely required hours.

V. WHAT THE LOGIC THEORIST PROVES

The Logic Theorist attempts to prove theorems in propositional logic using the five axioms of *Principia Mathematica*, Chapter 2. The notation used by the original authors is given in Table I.

TABLE I
LOGICAL NOTATION USED BY THE LOGIC THEORIST

Symbol	Meaning
I	Implication ($\rightarrow$)
V	Disjunction ($\vee$)
*	Conjunction ($\wedge$)
-	Negation ($\neg$)
=	Biconditional ($\leftrightarrow$)
. = .	Definitional equality

LT employs several proof methods:

- **Substitution**—substitute variables in an axiom or previously proved theorem to match the goal.
- **Detachment** (modus ponens)—from $A \rightarrow B$ and A , conclude B .
- **Forward chaining**—apply substitution and detachment working forward from known theorems.
- **Backward chaining**—work backward from the goal, seeking premises that would yield it.
- **Sublevel replacement**—replace a subexpression using a known equivalence.

Each proof attempt is bounded by an effort limit of 20,000 IPL machine cycles. This is a soft cap: it prevents the system

from *beginning* a new subproblem exploration once exceeded, but cannot interrupt an in-progress search.

Table II shows the results of the reanimated LT on the 23 theorems attempted. Of these, 16 are proved and 7 are not—results that are historically consistent with the original LT’s known behavior within its search effort limits.

TABLE II
THEOREMS ATTEMPTED BY THE REANIMATED LOGIC THEORIST

Thm.	Formula	Result
2.01	$(PI-P) I-P$	Proved
2.02	$QI (PIQ)$	Proved
2.04	$(PI (QIR)) I (QI (PIR))$	Proved
2.05	$(QIR) I ((PIQ) I (PIR))$	Proved
2.06	$(PIQ) I ((QIR) I (PIR))$	Proved
2.07	$PI (PVP)$	Proved
2.08	PIP	Proved
2.10	$-PVP$	Proved
2.11	$PV-P$	Proved
2.12	$PI--P$	Proved
2.13	$PV---P$	Proved
2.14	$--PIP$	Not proved
2.15	$(-PIQ) I (-QIP)$	Not proved
2.20	$PI (PVQ)$	Proved
2.21	$-PI (PIQ)$	Not proved
2.24	$PI (-PVQ)$	Proved
3.13	$(- (P*Q)) I (-PV-Q)$	Not proved
3.14	$(-PV-Q) I (- (P*Q))$	Not proved
3.24	$-(P*-P)$	Not proved
4.13	$P---P$	Not proved
4.20	$P=P$	Proved
4.24	$P= (P*P)$	Proved
4.25	$P= (PVP)$	Proved

A representative proof trace is shown below. Here LT proves Theorem 2.01, $(PI-P) I-P$:

2.01	$(PI-P) I-P$
PROOF FOUND.	
GIVEN	*1.2 (AVA) IA
SUBSTITUTION	.0 (PI-P) IP
SUBLEVEL REPL	2.01 (PI-P) I-P
Q.E.D.	
EFFORT	LIMIT 20000 ACTUAL 5579
SUBPROBLEMS	LIMIT 50 ACTUAL 1
SUBSTITUTIONS	LIMIT 50 ACTUAL 2

The proof begins from Axiom *1.2, (AVA) IA, substitutes $-P$ for A to obtain $(-PV-P) I-P$, then applies sublevel replacement (using the fact that P implies PVP) to transform the antecedent, yielding the desired theorem.

VI. DEBUGGING WITH LLMs: A COLLABORATION WITH AI TO REANIMATE AI

The most distinctive aspect of this project—and perhaps its most novel methodological contribution—was the essential role played by large language models as collaborative debugging partners. Over the course of approximately one year, I worked closely with two LLMs: Google’s Gemini (via the AntiGravity interface) and Anthropic’s Claude (via its terminal-based coding interface, Claude Code). The complete histories of these interactions are preserved in the project repository.

A. Teaching a Dead Language to an LLM

The collaboration began from a position of mutual ignorance. IPL-V is an essentially dead language: the manual is poorly OCR'd in places and obscure even where legible; the language itself is unlike any modern programming language; and there is no Stack Overflow, no GitHub corpus, no community of practitioners for the LLMs to have learned from. I was simultaneously learning the language myself while building the interpreter, which meant that the LLMs' primary source of information about IPL-V was my own Lisp code—which contained bugs.

This created a fascinating epistemological bootstrapping problem. The LLMs were inferring IPL-V semantics from my interpreter, sometimes correctly and sometimes not. When the interpreter had bugs, the LLMs would occasionally learn incorrect semantics and propagate those errors in their suggestions. Untangling this required constant vigilance about what was established knowledge versus what was inferred from possibly-buggy code.

B. The Two-Unknown Problem

A persistent challenge was what I came to think of as the two-unknown problem: when something went wrong, was the bug in my IPL-V interpreter, or in the transcribed LT code? Early on, I made a methodological decision that proved essential: we would assume the LT code was correct. This was not entirely justified—indeed, several transcription errors were eventually found—but it was a necessary simplifying assumption. Without it, every anomaly would have opened two lines of investigation, making progress nearly impossible.

This same problem was not lost on the original developers. In the McCorduck interviews, Cliff Shaw describes the particular difficulty of debugging heuristic programs: they do not crash, they do not produce obviously wrong output, they simply generate vast quantities of behavior that might or might not be correct. Sixty years later, with the same program, I faced the identical challenge.

C. The Wall—and How LLMs Broke Through It

The debugging arc had a clear inflection point. In the early phase, bugs manifested as crashes or obviously incorrect behavior—a J-function not implemented, a register not saved and restored, a stack underflow. These were frustrating but tractable. A human programmer, even working alone, could identify and fix them.

The second phase was qualitatively different. Once the interpreter was stable enough that LT ran without crashing, the output was thousands of lines of execution trace—machine cycles, register states, list operations—that *might* be correct. The program was heuristic; it was supposed to explore and backtrack. Distinguishing a correct backtrack from one caused by a subtle bug required understanding both the IPL-V machine semantics and LT's high-level proof strategy simultaneously, while tracking state across thousands of steps.

I will be frank: at this point, I had effectively given up. The cognitive load of holding the machine state in my

head across thousands of trace lines, while simultaneously questioning whether each line reflected correct or incorrect behavior, exceeded what I could sustain. The LLMs, however, were not inhibited by this. They could read thousands of lines of trace, insert diagnostic prints at ten-line intervals, and maintain the kind of tedious but precise state-tracking that the task demanded. It was enormously repetitive work that simultaneously required deep contextual understanding—a combination that, ironically, is precisely where current LLMs excel.

D. Simon's J's: The Breakthrough

A critical turning point came when the LLM had accumulated enough understanding of IPL-V—through months of working with my interpreter and the manual—that I was able to give it a new primary source: “Simon's J's” [9], a set of original IPL-V punched cards from 1962, preserved at the Computer History Museum. These cards contain Simon's own implementations of the J-functions in IPL-V assembly.

The cards had been transcribed by museum volunteers and made available online. When I provided these transcriptions to the LLM, it was able to compare Simon's original J-function implementations against my Lisp versions and identify discrepancies. This was a remarkable moment: an AI system, having learned a dead programming language primarily from a modern interpreter that it had helped debug, was now able to read original 1962 source code and use it to find bugs.

[PLACEHOLDER: A specific debugging anecdote will be inserted here, drawn from the Claude Code session logs. This will describe a particular J-function bug—the symptom in the LT trace, the discrepancy between the Lisp implementation and Simon's original, and how the LLM identified and resolved it.]

E. A Closer Collaboration Than Any Human

One observation deserves explicit mention: over the course of this project, my collaboration with the LLMs was closer than with any human engineer. This is not a slight to the human collaborators, whose contributions—particularly in transcription and encouragement—were essential. But the day-to-day, line-by-line work of understanding IPL-V semantics, reading traces, forming and testing hypotheses about bugs, and iterating on fixes was done in dialogue with Claude and Gemini. The LLMs served simultaneously as research assistants, sounding boards, and—once they had absorbed enough context—as something approaching genuine collaborators who understood the domain.

Both LLMs made substantive contributions. Gemini found one major bug; Claude found several others. In the end, I settled on working primarily with Claude, not because of a significant capability difference, but because it was easier to maintain a single long session with accumulated context than to switch between systems.

VII. REFLECTIONS: IPL, LISP, AND THE ARC OF AI

I have been programming in Lisp for fifty years. For most of that time, I attributed the genius of Lisp entirely to John

McCarthy. One of the most valuable outcomes of this project has been a revision of that understanding.

Building an IPL-V interpreter revealed that Newell, Simon, and Shaw had, by the late 1950s, already invented essentially every conceptual element that makes Lisp powerful: list manipulation, symbol processing, the idea that programs are themselves lists, dynamic memory allocation, recursion, higher-order functions, generators. What they lacked was the all-important homoiconic syntax—the unification of code and data in a single parenthesized notation that makes Lisp programs manipulable by Lisp programs.

This is not to diminish McCarthy’s contribution, which was genuinely brilliant. But it reframes the standard narrative. McCarthy’s genius was not in inventing list processing *ex nihilo*, but in finding the syntactic form that made these ideas—ideas that Newell, Simon, and Shaw had already developed—elegant and accessible. IPL-V and Lisp coexisted for approximately five years, but Lisp was so much simpler and more expressive that it rapidly supplanted its ancestor.

There is a personal dimension to this realization. I studied under both Newell and Simon at Carnegie Mellon University in the 1980s. IPL and LT were long since history by that time, and I never discussed either with them—we barely learned about these pioneering efforts in our classes. But something from working with these giants stuck with me. At CMU I was modeling scientific and engineering reasoning, and continued to do so for most of my career. Now, at the end of that career, I have been led back to try to understand how these brilliant scientist-engineers were inventing AI and cognitive science.

VIII. RELATED WORK

The reanimation of historical software systems has gained momentum in recent years. Most directly related is the ELIZA reanimation by Lane et al. [6], in which a team including the present author restored Weizenbaum’s original ELIZA to operation on a reconstructed CTSS running on an emulated IBM 7094. That project reconstructed the complete original hardware and operating system stack; the present work instead emulates the abstract machine specification in a modern language.

A 1968 paper describes a reimplementations of the Logic Theorist in LISP 1.5 [11]. That effort, however, rewrote LT’s logic in Lisp rather than faithfully emulating IPL-V and running the original code. The distinction is significant: a Lisp reimplementations demonstrates that LT’s algorithms can be expressed in Lisp (which is unsurprising), while the present work demonstrates that the original IPL-V code, as written by Newell, Simon, and Shaw’s team, still functions correctly when provided with a faithful implementation of the machine it was designed for.

To the author’s knowledge, no prior attempt has been made to build an IPL-V interpreter and execute original IPL-V programs.

IX. NEXT STEPS

The IPL-V interpreter developed for this project is not specific to the Logic Theorist. It implements the general IPL-V

abstract machine and can, in principle, run any IPL-V program. The most significant targets for future reanimation are the General Problem Solver (GPS) [12] and EPAM (Elementary Perceiver and Memorizer) [13], both originally written in IPL-V by members of the same research group. Archival work to locate complete source code for GPS is planned for spring 2026. Now that a proven IPL-V interpreter exists, running these programs should require substantially less effort than the initial construction of the interpreter itself.

The interpreter, the transcribed LT code, and the complete LLM interaction histories are available as open source at <https://github.com/jeffshrager/IPL-V>.

X. ACKNOWLEDGEMENTS

The following people contributed to this project in ways ranging from code transcription to weekly encouragement: Anthony Hay, Arthur Schwarz, Leigh Klotz, David M. Berry, Rupert Lane, Amy Majczyk, Ethan Ableman, and Paul McJones. I am particularly grateful to Hay, Schwarz, Klotz, and Berry, who received weekly progress updates throughout the project and responded with encouragement that was always helpful and sometimes technically substantive. Rupert Lane’s gridlock software caught several transcription errors that had survived multiple rounds of human review.

I also thank the Computer History Museum for preserving and making available “Simon’s J’s,” the original 1962 IPL-V J-function card deck, which proved essential during debugging.

Finally, I want to acknowledge Allen Newell and Herbert Simon, under whom I studied at CMU in the 1980s. We never discussed IPL or the Logic Theorist, but the intellectual values they instilled—rigor about mechanism, respect for the complexity of cognition, and the conviction that thinking could be understood computationally—led me, decades later, back to their earliest work.

REFERENCES

- [1] A. N. Whitehead and B. Russell, *Principia Mathematica*. Cambridge: Cambridge University Press, 1910–1913.
- [2] A. Newell, Ed., *Information Processing Language-V Manual*. Englewood Cliffs, NJ: Prentice-Hall, 1961.
- [3] J. McCarthy, “Recursive functions of symbolic expressions and their computation by machine, Part I,” *Communications of the ACM*, vol. 3, no. 4, pp. 184–195, Apr. 1960.
- [4] E. Stefferud, “The Logic Theory Machine: A Model Heuristic Program,” RAND Corporation, Santa Monica, CA, Memorandum RM-3731-CC, 1963.
- [5] A. Newell, Ed., *Information Processing Language-V Manual*, 2nd ed. Englewood Cliffs, NJ: Prentice-Hall, 1964.
- [6] R. Lane, A. Hay, A. Schwarz, D. M. Berry, and J. Shrager, “ELIZA Reanimated: Restoring the Mother of All Chatbots to One of the World’s First Time-Sharing Systems,” *IEEE Annals of the History of Computing*, 2025.
- [7] F. Gruenberger, “The History of the JOHNNIAC,” RAND Corporation, Santa Monica, CA, Memorandum RM-5654-PR, 1968.
- [8] P. McCorduck, *Machines Who Think*. San Francisco, CA: W. H. Freeman, 1979.
- [9] A. Bochanek, “Simon’s J’s,” Computer History Museum Blog, Sep. 26, 2012. [Online]. Available: <https://computerhistory.org/blog/simons-js/>
- [10] R. Lane, “Gridlock: Accurate code extraction from PDFs,” 2024. [Online]. Available: <https://github.com/rupertl/gridlock>
- [11] The Logic Theorist in LISP 1.5, *The Computer Journal*, vol. 11, no. 1, 1968.
- [12] A. Newell, J. C. Shaw, and H. A. Simon, “Report on a general problem-solving program,” in *Proc. International Conf. on Information Processing*, Paris, France, 1959, pp. 256–264.

- [13] E. A. Feigenbaum, "The simulation of verbal learning behavior," in *Proc. Western Joint Computer Conference*, 1961, pp. 121–132.